The Light Rider Cloud Platform:
Self-Serve Quantum Compute and Attested Entropy

Light Rider Inc. — Engineering

August 7, 2026 • Internal (Dev / Ops)

Companion to "The Light Rider Platform: Attested Multi-Source Entropy as a Service" (2026-07-14). Verified against the `cloud_platform_nextjs` source tree on 2026-08-07.

Abstract. The cloud platform is the customer-facing surface of Light Rider: a Next.js 16 application on Vercel, served from `platform.lightriderinc.com`, that lets a user run circuits on IQM quantum processors and draw attested entropy *without holding an IQM credential or operating the entropy backend*. It is a control-and-UI layer with a thin server API. Identity and roles come from Logto; the platform keeps Stripe-facing billing state — customers, subscriptions, a prepaid credit ledger, API keys, and a record of submitted jobs — while the lr / EMS backend calls back to check balances before running work. Every capability is served through server route handlers that attach the right upstream credential and call one of two services: the EMS entropy egress (the companion guide's subject) and iqm-proxy for circuit execution. The platform's defining job is to present those services as a metered, credentialed product while keeping their credentials server-side. This guide defines the platform's vocabulary, its architecture and request flows, its auth / billing / credential subsystems, how it consumes EMS, its deploy and operations model, and the work in progress.

Provenance. Endpoints, environment-variable names, backend ids, pricing, and the entropy source → policy map below are read from the code (routes, clients, `prisma/schema.prisma`, `.env.example`, `logto.ts`, the backend catalog, the Colab notebooks), not inferred. Items marked *[open]* are repo-flagged TODOs or operational details to settle, not gaps in this write-up.

Contents

1 Problem statement

Two upstream services do the real work, and both are awkward to consume directly. Running circuits on IQM means holding an IQM service token and building an IAM around it; drawing entropy from EMS means operating the backend, passing a privileged key, and verifying signed receipts. Neither is something a customer should have to do, and neither credential should ever reach a browser.

The platform resolves this by being a single hosted origin. A user signs in with Google or GitHub, receives an API key, and then runs circuits and draws entropy through **one domain**. The platform holds the privileged upstream credentials — the IQM service token (behind iqm-proxy) and the EMS key — **server-side only**, and re-exposes IQM's backends and EMS's pools and policies as product features. Compute is charged against a prepaid credit ledger; entropy is entitlement-gated at EMS.

Remark. Where state lives. *The Prisma schema is deliberately scoped to* Stripe-facing *state only (who the customer is in Stripe, what they are subscribed to, a credit ledger). It does not model quantum jobs, entropy pools, or usage enforcement — that lives in the lr / EMS backend, which is expected to call back into this app (or read a synced replica) to check balances before running work (see* `BILLING_INTEGRATION.md`*). The platform is the commerce and identity system of record; the backend is the compute system of record.*

Remark. No second trust root. *The platform adds no cryptographic guarantee of its own. Entropy attestation lives entirely in EMS's signed receipt, verified against the EMS public key; circuit results carry iqm-proxy's job identity. The platform's contribution is productization — identity, metering, keys, a coherent surface — not a second trust root.*

2 Preliminaries

This section defines the platform's vocabulary. It assumes the entropy concepts from the EMS guide (min-entropy, extractors, receipts, pools, policies) and adds the platform's own terms.

2.1 Runtime

Next.js App Router (v16 — a major with breaking changes; `AGENTS.md` tells contributors to read the bundled docs before writing code). React components render the UI; *route handlers* under `src/app/api/` are the server API and the only place secrets exist. **Vercel** hosts it; a push to `main` triggers a production build (§12).

2.2 Identity, billing, storage

Logto (`@logto/next`) owns sign-in, Google / GitHub OAuth, sessions, and the **Basic / Pro** roles. **Stripe** owns payments. The platform keeps a **credit ledger** (append-only, signed cents) for prepaid **Quantum Compute credits**, and separately reports **EaaS API usage** to a Stripe Billing Meter. **Prisma** is the ORM over **Postgres** (`DATABASE_URL` + `DIRECT_URL`; SQLite for local dev). **Supabase** appears only as object storage for user avatars.

2.3 Upstream services

EMS egress is reached at `EMS_BASE_URL`; the platform authenticates with a **Bearer** key that EMS resolves against tier entitlements. **iqm-proxy** is reached at `QUANTUM_PROXY_URL` — confirmed at `http://93.127.215.63` (iqm-proxy fronted by nginx on the bare IP, port 80). **device_instance routing** is a body field on each submission naming the QPU alias (`garnet`, `garnet:mock`, ...) that iqm-proxy's device registry expects. A **mock backend** is a `:mock` alias that runs on iqm-proxy's statevector simulator, free and unlimited.

Remark. *Several credentials share the `lr_` prefix and the word "key" — the user's platform API key, the quantum-proxy service key, and the legacy entropy token are all distinct. They are enumerated definitively in §7; read that before wiring anything.*

3 Architecture

One origin, strict client / server split. The React UI runs in the browser; route handlers run on Vercel and are the only place upstream credentials live. The browser never contacts EMS, iqm-proxy, Stripe's secret API, or the database directly — every such call is proxied through a same-origin route.

Concern	Technology	Notes
Web app + API	Next.js 16.2.9, React 19, TypeScript, Tailwind v4	UI + route handlers, one deployable
Hosting / CI	Vercel; semantic-release on `main`	push → build; release bot commits `CHANGELOG`
Identity	Logto (`@logto/next`)	sign-in via `auth.lightriderinc.com`; roles Basic / Pro
Billing	Stripe (`stripe` v22): Checkout, subs, webhooks, meter	test-mode until live is signed off
Database / ORM	Prisma 6 over Postgres (`DATABASE_URL`)	SQLite for local dev; Supabase = avatars
Entropy backend	EMS egress via `EMS_BASE_URL`	Bearer entitlement; `/v1/entropy/*`
Circuit exec	iqm-proxy via `QUANTUM_PROXY_URL`	`93.127.215.63`:80; six IQM backends + mock
Client SDK	`lightrider` (PyPI) v1.3.1; Colab notebooks	talks to the platform with an `lr_` key
PQC primitives	`@noble/post-quantum`, `@noble/hashes`	Vault (ML-KEM-768) / Signer (ML-DSA-65)

Table 1: Technology by concern (versions from `package.json`).

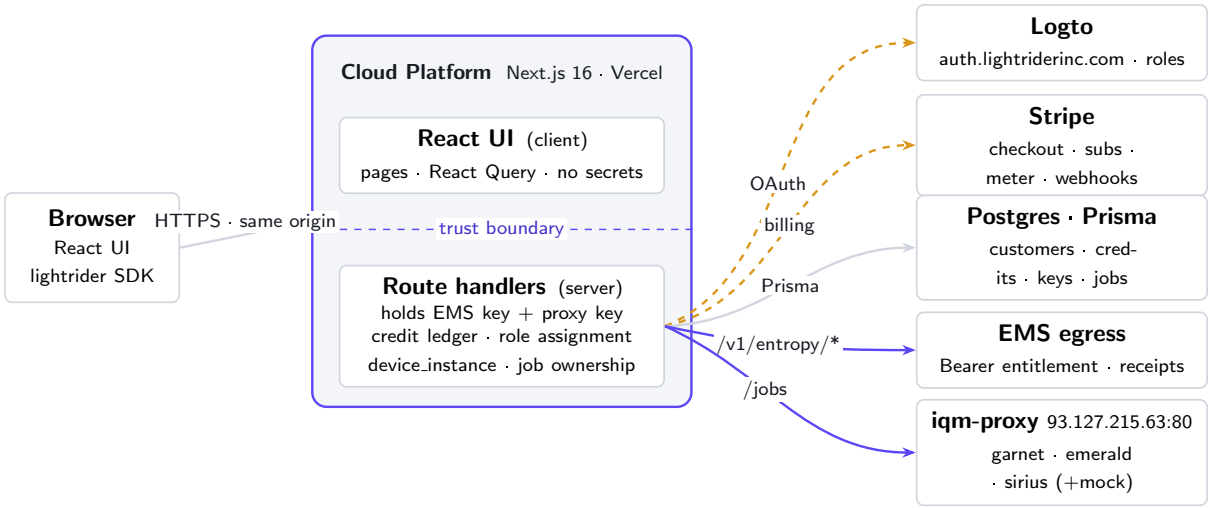

Figure 1: Platform overview. The browser talks to exactly one origin. Upstream credentials (the IQM token behind iqm-proxy, the EMS key) live only inside server route handlers and never cross the trust boundary. Indigo paths carry data (entropy bytes, circuit jobs); amber paths carry control (identity, billing). Not shown: per-vendor pass-through proxies (`/api/iqm`, `ibm`, `rigetti`, `nist`, `curby`, `drand`, `anu`) the catalog and legacy entropy path use.

4 Data flow

Three request families cover almost everything. Each begins in the UI or SDK and terminates in a server route handler that attaches the right credential and calls upstream.

4.1 Draw entropy

The main path is the EMS-backed console (`/entropy`). A separate legacy path serves the Quantum Vault "try a source" tab.

1. **UI request.** `requestEntropy()` (`src/lib/entropy/generate.ts`) maps the chosen source card to an EMS policy and calls `POST /api/ems/entropy` with `{mode:"single",policy, bytes}`. A multi-source draw sends `{mode:"multi",method,sourceIds,bytes}`.
2. **Server route.** `/api/ems/entropy` requires a signed-in Logto user (401 otherwise) and requires `EMS_BASE_URL` (500 otherwise).
3. **Call EMS.** It forwards to `EMS_BASE_URL` + `/v1/entropy/request` (single) or `/v1/entropy/multi` (multi), sending `Authorization: Bearer $EMS_API_KEY` and an (inert) `x-ems-api-key`.
4. **EMS responds.** `bytes_hex` plus the full signed receipt (sources, quality, credited min-entropy, `audit_event_id`, signature). The route returns EMS's body verbatim.
5. **Render + record.** The client keys a history entry on `receipt.request_id` and derives `createdAt` from `receipt.timestamp_unix_ns`.

```
POST $EMS_BASE_URL/v1/entropy/request        # or /v1/entropy/multi
Authorization: Bearer $EMS_API_KEY           # the real auth (entitlement-resolved)
x-ems-api-key: $EMS_API_KEY                  # inert until EMS grows a route that reads it
{ "bytes": 32, "policy": "quantum_verified",
  "application_id": "", "zone_id": "", "client_nonce": "" }
```

Listing 1: server route to EMS egress (from src/app/api/ems/entropy/route.ts)

Remark. Entitlement, not role. *EMS resolves the Bearer key against tier entitlements (`ems-egress/src/entitlement.rs`). An* anonymous *or under-entitled key is silently downgraded to the Free grant, which can only draw `pool_fastest` and never gets `multi_source`. The product intent is "no role/subscription gate on source selection," so `EMS_API_KEY` must be a fully-entitled key or signed-in users quietly lose sources (§14).*

Legacy path — `GET /api/entropy?source=\&bytes=`. Used by Quantum Vault's "My Keys" tab to try a source without EMS. `nist-beacon` hits NIST's real Randomness Beacon 2.0; otherwise, if `LR_TOKEN` is set, it runs an `h`-gate job on the LR quantum API and HKDF-derives bytes from `jobId + counts`; failing both, it returns `randomBytes` honestly labeled `X-Entropy-Source: csprng-fallback`. This is the one place a CSPRNG fallback still exists — the EMS console path never falls back.

4.2 Run a circuit

Circuit submission proxies to iqm-proxy with a shared service key; the customer never sees an IQM token.

1. **Submit.** UI or SDK sends `{backend,circuit,shots}` to `POST /api/lr/quantum/submit`, authenticated by a Logto session *or* an `Authorization: Bearer lr_...` key (`resolveCustomerFromRequest`). First-ever request lazily creates the Customer and grants the free signup credit.
2. **Gate on credits.** Cost = `ceil(shots * costPerShotCents)`. Real QPUs require having purchased credits at least once (→ `402 purchase_required`); then an insufficient balance (→ `402 insufficient_credits`). Mock backends cost 0, so both checks no-op.
3. **Route + call.** The handler resolves the backend to its `deviceInstance` and calls `QUANTUM_PROXY_URL + /jobs` with `Authorization: Bearer $QUANTUM_PROXY_SERVICE_KEY` and body `{circuit,shots,device_instance}`.
4. **Record atomically.** On success it writes a `QuantumJobSubmission` row and, if billable, decrements `creditsBalanceCents` + appends a negative `CreditLedgerEntry` — all in one `db.$transaction`.
5. **List + poll.** `GET /api/lr/quantum/jobs` returns the caller's own rows (cached status); the client re-polls live status per non-terminal job, and `finishedAt` is backfilled from iqm-proxy opportunistically.

```
# %pip install -q lightrider==1.3.1 requests
from lightrider import Circuit, get_backend

local = get_backend("statevector")           # free local sim, no key needed

base_url = "https://platform.lightriderinc.com"
session.headers["Authorization"] = f"Bearer {api_key}"   # lr_ key from /settings/keys

session.post(f"{base_url}/api/lr/quantum/submit",
    json={"backend": "iqm-garnet-mock", "circuit": circuit, "shots": 1000})
# poll {base_url}/api/lr/quantum/jobs/{id}  then  /result
```

Listing 2: what a customer runs (from docs/notebooks/quantum-quickstart-iqm-garnet.ipynb)

Two submission generations. `/api/lr/quantum/submit` (current) takes a full circuit object `{num_qubits,instructions:[{name,qubits,clbits?}]}`. The older `/api/lr/jobs` took `{gate,shots}` shorthand and is still used by the legacy entropy path and `src/lib/lr/client.ts`. Presets `h` / `bell` live in `src/lib/quantum/client.ts`.

4.3 Authenticate & meter

Sign-in, role assignment, and metering — the control plane over the two data flows.

1. **Sign in.** Logto (Google / GitHub via Light Rider's own OAuth apps) at `auth.lightriderinc.com`. New users are Basic.
2. **Subscribe.** Stripe Checkout for the Pro plan ($99/mo) or an API plan; a validation-only price (`STRIPE_PRICE_VALIDATION_TEST`) exists to exercise the whole pipeline end to end.
3. **Reconcile.** The webhook receiver handles `checkout.session.completed` and `customer.subscription.{created,updated,deleted}`, credits the ledger, and assigns the Logto Pro role (`LOGTO_PRO_ROLE_ID`).
4. **Guard idempotency.** Every processed event id is written to `ProcessedStripeEvent`, and ledger grants carry a unique `stripeEventId`.
5. **Meter.** Compute debits the ledger per job at submit; EaaS API overage is reported to a Stripe Billing Meter by the backend gateway (§6).

Remark. *Money state is authoritative in Stripe; the ledger and cached `creditsBalanceCents` are the platform's idempotent projection of it. Never let the client assert a balance.*

5 Authentication & identity

Identity is delegated to **Logto**. Sign-in uses **Google and GitHub OAuth** on Light Rider's own apps. Config lives in `src/app/logto.ts`: the sign-in endpoint (`auth.lightriderinc.com`) and `appId` are hardcoded there; `LOGTO_APP_SECRET`, `LOGTO_COOKIE_SECRET`, and the M2M credentials come from env. Two roles exist — **Basic** (default) and **Pro** — assigned from Stripe subscription state.

Two Logto footguns, both real in `logto.ts`.

1. **`baseUrl` falls back to localhost.** It is `process.env.NEXT_PUBLIC_BASE_URL || 'http://localhost:3000'`. If `NEXT_PUBLIC_BASE_URL` is not set for the deployed environment, OAuth callbacks resolve to localhost and sign-in breaks. Set it everywhere (preview and production).
2. **Management API cannot use the custom domain.** The Management API token endpoint and calls must go through the tenant's default `*.logto.app` domain (`managementEndpoint` is pinned to it), even though sign-in uses the custom domain. Do not merge the two into one host.

6 Billing & credits

Billing runs on **Stripe** and splits into three mechanisms. The Prisma schema models only the Stripe-facing side.

- **Idempotency.** `ProcessedStripeEvent` logs every handled event id; ledger grants carry a unique `stripeEventId`.
- **Webhook events.** `checkout.session.completed`, `customer.subscription.{created,updated,deleted}`; unknown events are recorded and ignored.
- **Mode.** Stripe defaults to test-mode in the example config, pending live-mode sign-off from Tony / Jake.

Remark. Pro is parked. *The V2 job path is* purely credit-gated *— there is no Pro check anywhere in `/api/lr/quantum/submit`. The Pro subscription flow, `isProCustomer()`, and `resync-pro-role` all still exist but are unreachable from submission today.*

Mechanism	How it works	Where
Prepaid Quantum Compute credits	Append-only `CreditLedgerEntry` (signed cents); cached `Customer.creditsBalanceCents`. Debited per job at submit — `ceil(shots * costPerShotCents)`. Free signup grant on first submit; real QPUs require ≥1 purchase.	`/api/lr/quantum/submit`, `lib/billing/`
Subscriptions	`Subscription` mirrors Stripe; `kind` is USER_PLAN or API_PLAN. User Pricing = Basic (free) + Pro ($99/mo, $99 credits); Starter / Developer / Professional documented but not offered; Enterprise → Contact Sales. API Pricing = Free / Starter ($99) / Developer ($499) / Business, usage-metered.	webhook, `lib/billing/plans.ts`
EaaS API usage meter	`reportApiUsage()` emits a Stripe Billing Meter event + records `ApiUsageEvent`. Designed to be called by the entropy API gateway (backend team), not this app.	`lib/billing/meter.ts`

Table 2: The three billing mechanisms.

7 Credentials

The most confusable part of the system. User-facing keys are created in `/settings/keys`: a key is `lr_live_` / `lr_test_` + 24 random bytes, stored as a **SHA-256** `keyHash` (Prisma `ApiKey`) with a short `keyPrefix`, and the plaintext is shown **once**. It is sent to the platform as a Bearer token and resolves to a `Customer`.

Credential	Direction	Storage / form
Platform API key	user → platform (Bearer)	Prisma `ApiKey`, SHA-256 hash, shown once; `lr_live_`/`lr_test_`
`EMS_API_KEY`	platform → EMS egress	env; `Authorization: Bearer` (+ inert `x-ems-api-key`). Must be fully entitled *[open]*
`QUANTUM_PROXY_SERVICE_KEY`	platform → iqm-proxy (Bearer)	env; an `lr_` key from iqm-proxy's own `/users`, one shared service account
`LR_TOKEN`	platform → LR quantum API (legacy)	env; Bearer, used only by `GET /api/entropy`. Parallel to the proxy key *[open]*
`IQM_TOKEN` · `RIGETTI_TOKEN` · `IBM_CLOUD_API_KEY`	platform → vendor APIs	env; used by the pass-through proxies `/api/iqm`, `rigetti`, `ibm`

Table 3: Every credential the platform touches.

Remark. *Because iqm-proxy sees only the one shared `QUANTUM_PROXY_SERVICE_KEY`, it cannot tell customers apart — per-customer job ownership is enforced locally by the `QuantumJobSubmission` table, which is what lets `jobs/[id]` and `/result` reject someone else's job id. The customer's `lr_` key and iqm-proxy's `lr_` keys share a prefix but are different namespaces; follow the endpoint to know which is which.*

8 Quantum compute

Six IQM backends are wired for submission (`src/lib/quantum/backends.ts`): **garnet, emerald, sirius**, each real and `:mock`. All post to iqm-proxy's `/jobs` with a `device_instance` string; the real ones cost **$0.002/shot** (`costPerShotCents: 0.2`, i.e. $2 per 1,000 shots), the mocks are free.

Backend id	device_instance	Cost
iqm-garnet / -mock	garnet / garnet:mock	$0.002/shot · free
iqm-emerald / -mock	emerald / emerald:mock	$0.002/shot · free
iqm-sirius / -mock	sirius / sirius:mock	$0.002/shot · free

Table 4: The submission-wired backends.

- **Routing.** `device_instance` tells iqm-proxy's device registry which alias to use; without it, iqm-proxy falls back to its single hardcoded `IQM_SERVER` alias.
- **Catalog vs wired.** The `/backends` catalog lists IQM + Rigetti + IBM (via `useIqmBackends` / `useIbmBackends` / `useRigettiBackends` and the per-vendor proxies), but only the six IQM machines can be submitted to today. `src/data/backends.ts` also holds placeholder "Lightrider Aurora / Vega / Nova" cards for layout — their per-second pricing is display placeholder, not the billing path.
- **Circuit format.** Full circuit JSON `{num_qubits,instructions}`; presets `h` / `bell` in `quantum/client.ts`; the free local `statevector` sim runs in the SDK with no key.
- **Jobs.** `/jobs` shows in-app and SDK / Colab submissions identically (same `QuantumJobSubmission` rows, same resolved customer), with ownership enforced locally.

9 Entropy integration (consuming EMS)

The entropy console (`/entropy`) is an EMS client. Each UI source card maps to an EMS *policy* that routes to a tier pool; the pool's live contributors come back in the receipt's `contributing_sources`. The mapping is in `src/lib/entropy/generate.ts`.

UI source	Policy sent	Pool
nist-beacon	cost_optimized	pool_fastest
rdseed	fastest_available	pool_fastest
curby	quantum_verified	pool_quantum_verified
iqm-resonance	hybrid_mix	pool_quantum_verified
(any other / default)	fastest_available	pool_fastest

Table 5: Source card → EMS policy → pool (verbatim from `POLICY_BY_SOURCE_ID`).

The source selector also surfaces **Drand** and **ANU Quantum RNG**; these are not in the policy map, so through the EMS route they fall to the default policy — but both also have dedicated pass-through proxies (`/api/drand`, `/api/anu`). CURBy's card notes its quantum beacon is temporarily on a classical fallback, and IQM Resonance is described as an IQM mock simulator.

Remark. Verify live membership. *Policy names here (`cost_optimized`, `hybrid_mix`) extend the three in the EMS guide, and `pool_highest_quality` is not referenced by the UI at all. The card copy*

is the platform's presentation; a receipt's real `contributing_sources` *is decided by EMS at serve time. Reconcile the two before making any source-quality claims to customers.*

The receipt shape the platform consumes (`EntropyReceipt` in `generate.ts`) matches the EMS guide's Table 2 plus an `audit_event_id` field.

10 Applications

The `/applications` grid ships four cards (`src/components/applications/ApplicationsGrid.tsx`), all powered by platform entropy / PQC primitives:

- **True random dice roll** *[Demo]* — a dice roller driven by quantum entropy.
- **Secure password generator** *[Demo]* — strong passwords from quantum randomness.
- **Quantum Vault** *[New]* — encrypts a secret with **ML-KEM-768 + AES-256-GCM** (`@noble/post-quantum`, `lib/pqc/quantumVault.ts`); its "My Keys" tab uses the legacy `/api/entropy` try-a-source path.
- **Quantum-Safe Signer** *[New]* — signs any text with **ML-DSA-65**.

Placeholders for further apps (prioritization, geodata, benchmark, readiness canvas) exist in the modal switch but are not rendered yet. **Colab quickstarts** ship in `docs/notebooks/`: a generic one plus per-backend notebooks for Garnet / Emerald / Sirius, all pinned to `lightrider==1.3.1` and pointed at the platform origin.

Remark. *Synthetic data and the Octopus / QDash ops platform are* not *in this repo — synthetic generation lives in the* `lightrider` *package (EMS guide §12), and Octopus is a separate inheritance (§14). Do not document them as shipped platform surfaces here.*

11 Repository layout

The real tree (`src/`), from the repo. One Next.js App Router project.

12 Build, deploy, and operations

12.1 Build

Prisma client generation precedes the Next build. Local DB work uses `prisma migrate dev`; the Stripe webhook can be exercised locally with the `stripe:listen` script.

```
build         : prisma generate && next build
db:migrate    : prisma migrate dev
stripe:listen : stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

Listing 3: package.json scripts

12.2 Deploy the platform

All work is on `main`. semantic-release runs on `main` and its `@semantic-release/git` step commits `CHANGELOG.md` + `package.json` back with a `chore(release): ... [skip ci]` message — so `origin/main` moves under you and you must pull before pushing. A push to `main` triggers Vercel.

```
# the release bot commits to main - always pull first
git pull origin main --no-rebase
git push origin main
# set NEXT_PUBLIC_BASE_URL per environment or Logto callbacks break (see 5)
```

Path	Contents
src/app/api/ems/entropy	EMS-backed entropy proxy (single + multi), auth-gated
src/app/api/entropy	legacy try-a-source: NIST Beacon / LR job / CSPRNG fallback
src/app/api/lr/quantum/{submit,jobs/[id]/result}	circuit submit, job list, per-job status and result
src/app/api/{iqm,ibm,rigetti,nist,curby,drand,anu}/[...path]	per-vendor pass-through proxies
src/app/api/billing/*	checkout, portal, usage, cancel/reactivate, resync-pro-role
src/app/api/webhooks/stripe	Stripe webhook receiver (idempotent)
src/app/api/settings/tokens/{,rotate}	create / rotate the user's platform API key
src/app/{entropy,jobs,backends,applications,settings}	the user-facing pages
src/app/logto.ts + callback + actions	Logto config (the baseUrl caveat), OAuth callback, sign-in
src/lib/entropy/*	generate.ts (source-to-policy map, receipt type), beacon, health
src/lib/quantum/*	backends.ts (device_instance + pricing), submit/poll client
src/lib/{lr,iqm,ibm,rigetti,nist,curby,drand,anu}/client.ts	upstream HTTP clients
src/lib/billing/*	db, customer, plans, meter, planCheck, ownership
src/lib/auth/*	apiKeys.ts (SHA-256), session.ts, resolveCustomer.ts
src/lib/{pqc,stripe,supabase,logto,tour}/*	ML-KEM/DSA vault, Stripe, avatars, Logto mgmt, tour
prisma/schema.prisma	Customer, Subscription, CreditLedgerEntry, ApiKey, QuantumJobSubmission, ...
docs/notebooks/	Colab quickstarts (generic + garnet/emerald/sirius), SDK 1.3.1

Table 6: Directory map.

Listing 4: deploy (Vercel builds on push to main)

12.3 Deploy iqm-proxy

iqm-proxy runs on `93.127.215.63` behind nginx (:80), reached via `QUANTUM_PROXY_URL`. It is deployed by copying the FastAPI app to the server; because it is edited on Windows, strip CRLF first.

```
sed -i 's/\r$//' iqm-proxy/*.py              # strip Windows CRLF
scp -r iqm-proxy spoo2@93.127.215.63:/opt/iqm-proxy/
ssh spoo2@93.127.215.63 'cd /opt/iqm-proxy && <restart service>'   # unit/path: confirm
```

Listing 5: deploy iqm-proxy to 93.127.215.63

12.4 Servers & access

EMS and iqm-proxy share `93.127.215.63`. SSH is key-based: `spoo2` (sudo) and `varun` (`700` on `.ssh`, `600` on `authorized_keys`).

Remark. Docker group is root. *Membership in the `docker` group is root-equivalent; settle Varun's group assignments explicitly rather than by default.*

13 Configuration reference

Every variable below is read from `.env.example` / `logto.ts`. No values — names and purpose only. Configure them in Vercel (production).

Variable	Purpose
`IQM_TOKEN`	IQM Resonance token for the `/api/iqm` proxy.
`RIGETTI_TOKEN`	Optional Rigetti QCS token (calibration works without it).
`IBM_CLOUD_API_KEY` · `IBM_INSTANCE_CRN`	IBM Quantum; proxy exchanges the key for a short-lived IAM token. Optional: `IBM_API_VERSION`, `IBM_QUANTUM_BASE_URL`.
`EMS_BASE_URL`	EMS egress origin (no trailing slash). Required — `/api/ems/entropy` 500s if unset.
`EMS_API_KEY`	Sent as Bearer (+ inert `x-ems-api-key`). Provision with full entitlements (§14).
`SUPABASE_URL` · `SUPABASE_SERVICE_ROLE_KEY` · `SUPABASE_AVATAR_BUCKET`	Avatar storage only. Service-role key is server-only.
`DATABASE_URL` · `DIRECT_URL`	Prisma. SQLite for local dev; Postgres for staging / prod. `DIRECT_URL` for migrations.
`STRIPE_SECRET_KEY` · `STRIPE_PUBLISHABLE_KEY` · `STRIPE_WEBHOOK_SECRET`	Stripe API + webhook signature. Test-mode until sign-off.
`STRIPE_PRICE_USER_*` · `STRIPE_PRICE_API_*` · `STRIPE_METER_EVENT_NAME` · `STRIPE_PRICE_VALIDATION_TEST`	Price ids per tier (only `USER_PRO` offered), meter event name, validation-flow price.
`LOGTO_APP_SECRET` · `LOGTO_COOKIE_SECRET`	Logto app secret; 32+ char cookie encryption key.
`LOGTO_M2M_APP_ID` · `LOGTO_M2M_APP_SECRET` · `LOGTO_PRO_ROLE_ID`	Management-API M2M app; Pro role id.
`NEXT_PUBLIC_BASE_URL`	Drives Logto `baseUrl`. Must be set per environment (§5).
`QUANTUM_PROXY_URL` · `QUANTUM_PROXY_SERVICE_KEY`	iqm-proxy origin (`http://93.127.215.63`) and its shared `lr_` service key.
`LR_BASE_URL` · `LR_TOKEN`	Legacy — used only by `GET /api/entropy`. Overlaps the proxy vars above *[open]*.

Table 7: Platform environment variables.

14 Work in progress

Open items, grounded in what the code and comments flag:

- **Provision a fully-entitled `EMS_API_KEY`.** An under-entitled key silently limits signed-in

users to `pool_fastest` and blocks `multi_source` — the opposite of the "no gate on source selection" intent (§9).

- `x-ems-api-key` **is inert** until EMS grows a route that reads it — flagged to Martin's side as the iqm-proxy `lr_`-key/IAM pattern.
- **Stripe live-mode cutover** — test-mode until Tony / Jake confirm.
- **Wire Rigetti + IBM for submission** — catalog-only today; only the six IQM backends can be run.
- **Replace the placeholder** `/backends` **catalog** (`src/data/backends.ts` Aurora / Vega / Nova) with the live backends API.
- **Reconcile Drand / ANU** into the EMS policy map, or keep them on their dedicated proxies deliberately (§9).
- **EaaS API metering handoff** — the backend gateway needs to call `reportApiUsage()` / this app (see `BILLING_INTEGRATION.md`).
- **Resolve the** `LR_TOKEN` **vs** `QUANTUM_PROXY_SERVICE_KEY` **drift** in the legacy entropy path.
- **Un-park Pro** if / when Pro-gated features return (the flow exists but is unreachable from submit).
- **Roadmap, not in this repo:** Octopus / QDash for Pro users; a synthetic-data surface over the `lightrider` package.

15 Glossary

Term	Meaning
Route handler	Server-only function under `src/app/api/`; the only place upstream credentials exist.
Credit ledger	Append-only `CreditLedgerEntry` of prepaid Quantum Compute credits (signed cents).
`costPerShotCents`	Per-backend price; real IQM = 0.2 ($0.002/shot), mock = 0.
`device_instance`	Body field naming the QPU alias (`garnet`, `garnet:mock`) for iqm-proxy's registry.
Mock backend	`:mock` alias on iqm-proxy's statevector simulator — free, unlimited.
`EMS_BASE_URL`	EMS egress origin; the platform calls `/v1/entropy/*` there.
Entitlement	EMS's Bearer-key tier resolution; anonymous → Free → `pool_fastest` only.
Policy	EMS selection policy the console sends (`cost_optimized`, `fastest_available`, `quantum_verified`, `hybrid_mix`).
Receipt	EMS's signed attestation; `EntropyReceipt` mirrors it (+ `audit_event_id`).
Platform API key	`lr_live_`/`lr_test_` Bearer token; SHA-256 `ApiKey.keyHash`; shown once.
`QUANTUM_PROXY_SERVICE_KEY`	Shared `lr_` service-account key to iqm-proxy; ownership enforced locally.
`LR_TOKEN`	Legacy Bearer for `GET /api/entropy`'s quantum-job entropy path.
Quantum Vault / Signer	Apps using ML-KEM-768 + AES-256-GCM / ML-DSA-65 (`@noble/post-quantum`).
`QuantumJobSubmission`	Local row per submitted job; the per-customer ownership record.
`ProcessedStripeEvent`	Idempotency log of handled Stripe event ids.
semantic-release	CI on `main`; auto-commits `CHANGELOG` / `package.json` — hence pull-before-push.